

Sistemas Expertos – Parte DOS

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá

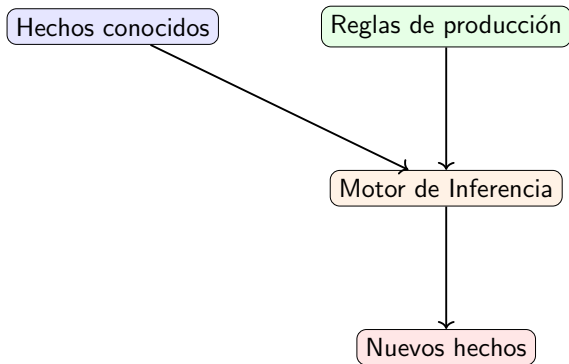


17 de febrero de 2026

¿Qué es un Motor de Inferencia?

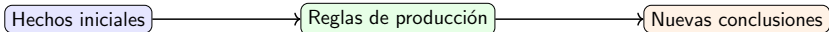
- ▶ Es el **componente central** del sistema experto responsable del razonamiento lógico.
- ▶ **Aplica reglas de la base de conocimientos a los hechos disponibles para deducir nuevos hechos.**
- ▶ Implementa mecanismos de inferencia como:
 - ▶ **Encadenamiento hacia adelante:** parte de los hechos y aplica reglas hasta alcanzar una conclusión.
 - ▶ **Encadenamiento hacia atrás:** parte de una hipótesis o meta y trabaja hacia atrás para verificar si puede derivarse.

Ejemplo: Si la base de hechos contiene A y $A \rightarrow B$, entonces el motor infiere B .

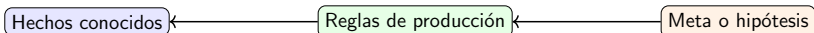


Tipos de Inferencia.

- ▶ **Encadenamiento hacia adelante:** parte de los hechos disponibles y aplica reglas para deducir nuevas conclusiones.
 - ▶ Ejemplo: $A, A \rightarrow B \Rightarrow$ se infiere B .



- ▶ **Encadenamiento hacia atrás:** parte de una hipótesis o meta y trabaja hacia atrás buscando hechos que la validen.
 - ▶ Ejemplo: Quiero demostrar B . ¿Existe A tal que $A \rightarrow B$ y se conoce A ?

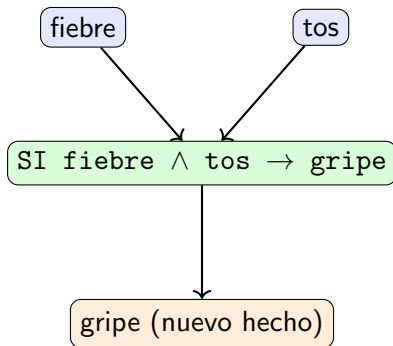


Encadenamiento hacia Adelante.

- ▶ Parte de un conjunto inicial de **hechos**.
- ▶ Se buscan y aplican reglas cuyo **antecedente** coincide con los hechos.
- ▶ Se generan nuevos hechos (**consecuentes**) y se repite el proceso hasta alcanzar una conclusión.

Ejemplo:

- ▶ **Hechos iniciales:** fiebre, tos
- ▶ **Regla:** SI fiebre $\wedge$ tos $\rightarrow$ gripe
- ▶ **Resultado:** se infiere gripe

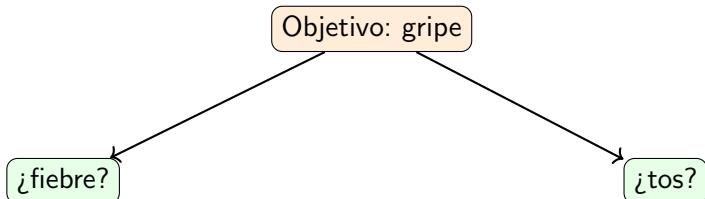


Encadenamiento hacia Atrás.

- ▶ Se parte de una **meta u objetivo** que se desea verificar.
- ▶ Se buscan reglas cuyo **consecuente** coincida con el objetivo.
- ▶ Los **antecedentes** de la regla se convierten en nuevas sub-metas que deben ser verificadas.

Ejemplo:

- ▶ **Objetivo:** gripe
- ▶ **Regla:** SI fiebre $\wedge$ tos $\rightarrow$ gripe
- ▶ ¿Se puede probar fiebre? ¿Se puede probar tos?



Si ambos se verifican, entonces el objetivo se cumple.

Ejemplo en Python: Motor de Inferencia.

Objetivo

- ▶ Comprender el encadenamiento hacia adelante como un proceso para obtener nuevos hechos a partir de los conocidos, aplicando reglas paso a paso.
- ▶ Entender el encadenamiento hacia atrás como una forma de comprobar si un objetivo se puede alcanzar, buscando reglas que lo justifiquen y verificando sus condiciones.

- ▶ **Ver Notebook.**

Manejo de Incertidumbre.

- ▶ En SE, el conocimiento puede ser **incompleto, incierto o impreciso**.
- ▶ Se usan técnicas para razonar en presencia de incertidumbre:
- ▶ **Factores de certeza (Certainty Factors):**
 - ▶ Utilizados en sistemas como *MYCIN*.
 - ▶ Expresan la confianza en una inferencia:
 $CF(\text{Hipotesis}, \text{Evidencia}) \in [-1, 1]$.
 - ▶ Ejemplo: $CF(\text{gripe}, \text{fiebre}) = 0,7$
- ▶ **Probabilidad Bayesiana:**
 - ▶ Modelo probabilístico: $P(H|E) = \frac{P(E|H)P(H)}{P(E)}$
 - ▶ Ejemplo: ¿Cuál es la probabilidad de tener gripe dado que hay fiebre?
- ▶ **Lógica Difusa:**
 - ▶ Permite grados de pertenencia en vez de valores binarios.
 - ▶ Ejemplo: $\mu_{\text{fiebre alta}}(38,5^\circ \text{C}) = 0,8$

Factores de Certeza (Certainty Factors).

- ▶ Modelo usado para manejar la incertidumbre en sistemas expertos como MYCIN.
- ▶ Cada regla y hecho tiene asociado un **Factor de Certeza (FC)** en el rango $[-1, 1]$.
 - ▶ FC positivo: evidencia a favor.
 - ▶ FC negativo: evidencia en contra.
 - ▶ FC cercano a 0: incertidumbre alta.
- ▶ El FC de una **conclusión** se calcula como el producto del FC de la regla y el mínimo FC de las premisas.

Ejemplo:

Regla: SI fiebre $\wedge$ tos $\rightarrow$ gripe

$$FC_{\text{regla}} = 0,8, \quad FC_{\text{fiebre}} = 0,9, \quad FC_{\text{tos}} = 0,7$$

$$FC_{\text{conclusión}} = FC_{\text{regla}} \cdot \min(FC_{\text{fiebre}}, FC_{\text{tos}}) = 0,8 \cdot 0,7 = 0,56$$

Ejemplo en Python: Factores de certeza.

Objetivo

Comprender cómo un sistema experto puede inferir diagnósticos a partir de síntomas con incertidumbre, utilizando factores de certeza para expresar distintos niveles de confianza en las conclusiones, y explorando posibles extensiones como reglas negativas, umbrales de decisión y justificación de las inferencias.

► **Ver Notebook.**

Contradicciones en el Motor de Inferencia.

- ▶ Las contradicciones ocurren cuando diferentes reglas producen **conclusiones incompatibles**.
- ▶ Estas situaciones pueden surgir debido a:
 - ▶ Conocimiento conflictivo en la base de reglas.
 - ▶ Incertidumbre o evidencia parcial contradictoria.
- ▶ Métodos comunes para resolver contradicciones:
 - ▶ **Priorización de reglas:** se asigna una jerarquía a las reglas y prevalecen las de mayor prioridad.
 - ▶ **Resolución por votación:** se consulta a múltiples fuentes o expertos y se elige la opción con mayor consenso.
 - ▶ **Lógica no monótona:** permite retractar conclusiones cuando nueva información contradice las anteriores.

Teoría de la Probabilidad.

- ▶ Asocia eventos con valores numéricos entre 0 y 1, indicando su grado de certeza.
- ▶ **Probabilidad condicional:**

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (\text{probabilidad de } A \text{ dado que ocurrió } B)$$

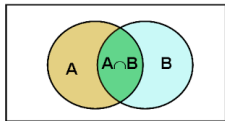


Imagen tomada de:
[Internet.](#)

Ejemplo: lanzamiento de un dado.

- ▶ Espacio muestral: $\Omega = \{1, 2, 3, 4, 5, 6\}$, entonces $n(\Omega) = 6$.
- ▶ Evento A: sacar un número par.
 $A = \{2, 4, 6\} \Rightarrow n(A) = 3 \Rightarrow P(A) = \frac{n(A)}{n(\Omega)} = \frac{3}{6} = 0,5$
- ▶ Evento B: sacar un número mayor que 3.
 $B = \{4, 5, 6\} \Rightarrow n(B) = 3 \Rightarrow P(B) = \frac{n(B)}{n(\Omega)} = \frac{3}{6} = 0,5$
- ▶ Intersección $A \cap B$: $A \cap B = \{4, 6\} \Rightarrow n(A \cap B) = 2 \Rightarrow$
 $P(A \cap B) = \frac{n(A \cap B)}{n(\Omega)} = \frac{2}{6} = \frac{1}{3} \Rightarrow P(A|B) = \frac{2}{3}.$

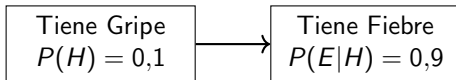
Regla de Bayes.

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

- ▶ Útil para actualizar creencias ante nueva evidencia.

Ejemplo:

- ▶ **Hipótesis:** H = “El paciente tiene gripe”
- ▶ **Evidencia:** E = “El paciente tiene fiebre”



- ▶ Esta red representa la relación causal: tener gripe puede causar fiebre.

- ▶ Se puede calcular la probabilidad inversa $P(H|E)$ usando la Regla de Bayes.
- ▶ Suposiciones:

$$P(H) = 0,1, \quad P(E|H) = 0,9, \quad P(E) = 0,2$$

- ▶ Aplicando la regla de Bayes:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{0,9 \cdot 0,1}{0,2} = 0,45$$

- ▶ **Interpretación:** dado que el paciente tiene fiebre, la probabilidad de que tenga gripe aumenta a 45 %.

Ejemplo en Python: Basado en Probabilidad Bayesiana.

Objetivo

Modelar un sistema experto que pueda inferir enfermedades a partir de síntomas, manteniendo la incertidumbre bajo control probabilístico.

► **Ver Notebook.**

Lógica Difusa (Fuzzy Logic).

- ▶ Propuesta por Lotfi Zadeh en 1965.
- ▶ Ideal para modelar incertidumbre lingüística: “temperatura alta”, “presión baja”, etc.
- ▶ Se basa en conjuntos difusos con funciones de pertenencia.

Función de pertenencia:

$$\mu_A(x) : X \rightarrow [0, 1]$$

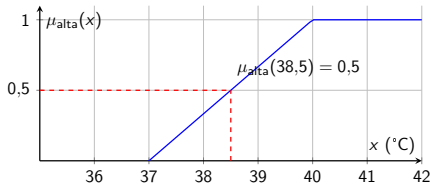
- ▶ x es un valor del universo de discurso X .
- ▶ $\mu_A(x)$ indica el grado de pertenencia de x al conjunto difuso A .

Ejemplo: temperatura corporal. Conjunto difuso: "Temperatura Alta"

$$\mu_{\text{alta}}(x) = \begin{cases} 0 & \text{si } x \leq 37 \\ \frac{x - 37}{3} & \text{si } 37 < x < 40 \\ 1 & \text{si } x \geq 40 \end{cases}$$

Ejemplo: Para
 $x = 38,5^{\circ}\text{C}$, se tiene:

$$\mu_{\text{alta}}(38,5) = \frac{38,5 - 37}{3} = 0,5$$



Aplicación de Reglas Difusas.

Forma general:

Si x es A ENTONCES y es B

Ejemplo práctico:

- ▶ **Regla:** Si la temperatura es “alta”, entonces la probabilidad de “gripe” es “alta”.
- ▶ **Inferencia:** Si $\mu_{\text{alta}}(x) = 0,5$, entonces $\mu_{\text{gripe}} = 0,5$.

Ejemplo en Python: Basado en Lógica difusa.

Objetivo

Presentar un sistema de inferencia difusa aplicado a un problema de diagnóstico, que permite manejar la incertidumbre mediante variables difusas y reglas tipo if-then.

► **Ver Notebook.**

Redes Bayesianas.

Definición:

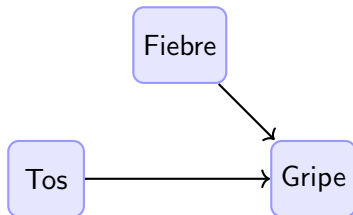
- ▶ Modelo gráfico probabilístico dirigido (grafo acíclico).
- ▶ Representan relaciones de dependencia condicional entre variables.
- ▶ Cada nodo representa una variable aleatoria.
- ▶ Cada arco indica una dependencia condicional.

Factorización conjunta:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i))$$

- ▶ $\text{Pa}(X_i)$ (abreviatura de *Parents of X_i*) representa el conjunto de variables padres de la variable X_i en la red bayesiana.
- ▶ Es decir, las variables de las cuales depende directamente X_i .

Estructura de una Red Bayesiana.



Interpretación:

- ▶ Gripe depende condicionalmente de Tos y Fiebre.
- ▶ Tos y Fiebre son independientes a priori.

$$P(\text{Tos}, \text{Fiebre}, \text{Gripe}) = P(\text{Tos}) \cdot P(\text{Fiebre}) \cdot P(\text{Gripe} \mid \text{Tos}, \text{Fiebre})$$

Ejemplo.

Datos (distribuciones) conocidos:

- ▶ $P(\text{Tos} = \text{sí}) = 0,3 \Rightarrow P(\text{Tos} = \text{no}) = 0,7$
- ▶ $P(\text{Fiebre} = \text{sí}) = 0,4 \Rightarrow P(\text{Fiebre} = \text{no}) = 0,6$
- ▶ $P(\text{Gripe} = \text{sí} \mid \text{Tos}, \text{Fiebre})$ dado por la tabla:

Tos	Fiebre	$P(\text{Gripe} = \text{sí})$
Sí	Sí	0.95
Sí	No	0.60
No	Sí	0.55
No	No	0.10

Caso práctico: Se conoce:

$$P(\text{Gripe} \mid \text{Tos} = \text{sí}, \text{Fiebre} = \text{sí}) = 0,95$$

Un paciente presenta tos y fiebre.

$\Rightarrow$ Alta probabilidad de gripe.

Con información parcial:

► Si solo se observa Fiebre, se puede calcular:

$$P(\text{Gripe} \mid \text{Fiebre}) = \sum_{\text{Tos}} P(\text{Gripe} \mid \text{Fiebre}, \text{Tos}) \cdot P(\text{Tos})$$

Cálculo:

$$\begin{aligned} P(\text{Gripe} = \text{sí} \mid \text{Fiebre} = \text{sí}) &= P(\text{Gripe} = \text{sí} \mid \text{Fiebre} = \text{sí}, \text{Tos} = \text{sí}) \cdot P(\text{Tos} = \text{sí}) \\ &\quad + P(\text{Gripe} = \text{sí} \mid \text{Fiebre} = \text{sí}, \text{Tos} = \text{no}) \cdot P(\text{Tos} = \text{no}) \\ &= 0,95 \cdot 0,3 + 0,55 \cdot 0,7 \\ &= 0,285 + 0,385 = \boxed{0,67} \end{aligned}$$

Probabilidades conjuntas:

$$P(\text{Tos, Fiebre, Gripe} = \text{sí}) = P(\text{Tos}) \cdot P(\text{Fiebre}) \cdot P(\text{Gripe} = \text{sí} \mid \text{Tos, Fiebre})$$

Tos	Fiebre	$P(\text{Gripe} = \text{sí} \mid \cdot)$	$P(\text{Tos, Fiebre, Gripe} = \text{sí})$
Sí	Sí	0.95	$0,3 \cdot 0,4 \cdot 0,95 = \mathbf{0,114}$
Sí	No	0.60	$0,3 \cdot 0,6 \cdot 0,60 = \mathbf{0,108}$
No	Sí	0.55	$0,7 \cdot 0,4 \cdot 0,55 = \mathbf{0,154}$
No	No	0.10	$0,7 \cdot 0,6 \cdot 0,10 = \mathbf{0,042}$

Probabilidad total de gripe:

$$P(\text{Gripe} = \text{sí}) = 0,114 + 0,108 + 0,154 + 0,042 = \boxed{0,418}$$

Ejemplo en Python: Basado en Redes Bayesianas.

Objetivo

Demostrar cómo un sistema experto puede manejar la incertidumbre utilizando una red bayesiana para diagnosticar la probabilidad de una enfermedad (gripe) a partir de síntomas observados (fiebre y dolor de cabeza), modelando las relaciones causales y probabilísticas entre variables.

► **Ver Notebook.**

Técnicas para Combinar Evidencias.

- ▶ **Factor de certeza (FC): MYCIN.**

$$FC_{\text{nueva}} = FC_{\text{previa}} + (1 - FC_{\text{previa}}) \cdot FC_{\text{nueva}}$$

- ▶ Dadas dos reglas que contribuyen a la hipótesis H :
 - ▶ Regla 1: $FC_1 = 0,7$
 - ▶ Regla 2: $FC_2 = 0,5$

Combinación:

$$FC = 0,7 + (1 - 0,7) \cdot 0,5 = 0,7 + 0,15 = 0,85$$

Ejemplo en Python: Técnicas para Combinar Evidencias.

Objetivo

Ilustrar cómo un sistema experto puede manejar la incertidumbre y combinar evidencias parciales mediante la técnica del Factor de Certeza (FC), utilizada en el sistema MYCIN.

► **Ver Notebook.**

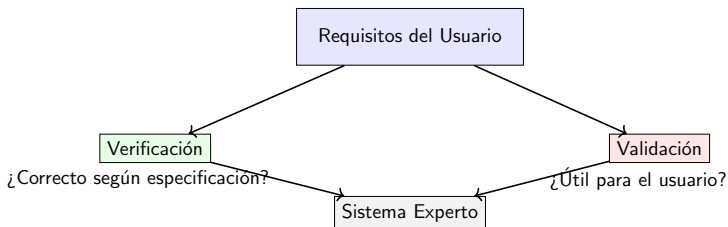
Resumen.

- ▶ El manejo de incertidumbre es fundamental para que los sistemas expertos puedan operar en entornos reales y complejos.
- ▶ Herramientas formales utilizadas:
 - ▶ **Teoría de la probabilidad:** modela eventos inciertos mediante distribuciones numéricas.
 - ▶ **Lógica difusa:** permite trabajar con grados de verdad intermedios entre 0 y 1.
 - ▶ **Redes bayesianas:** representan relaciones probabilísticas entre variables mediante grafos dirigidos.
- ▶ Estas técnicas permiten representar y razonar con conocimiento:
 - ▶ Incompleto
 - ▶ Ambiguo
 - ▶ Parcial o contradictorio

Verificación y Validación.

Los sistemas expertos deben pasar por un proceso riguroso para garantizar su **fiabilidad** y **utilidad**.

- ▶ **Verificación:** asegura que el sistema ha sido construido correctamente conforme a sus especificaciones.
- ▶ **Validación:** asegura que el sistema cumple con los requisitos del usuario y resuelve el problema real.



- ▶ **Verificación:** “¿Hemos construido el sistema correctamente?”
 - ▶ Evalúa la **correspondencia** entre la implementación y el diseño.
 - ▶ Técnicas: revisión de reglas, pruebas unitarias, detección de ciclos o contradicciones.
- ▶ **Validación:** “¿Hemos construido el sistema correcto?”
 - ▶ Evalúa si el sistema satisface los **requisitos del usuario**.
 - ▶ Técnicas: pruebas con casos reales, evaluación por expertos humanos, comparación con decisiones estándar.

Aspecto	Verificación	Validación
Pregunta clave	“¿Está bien construido?”	“¿Hace lo que debe?”
Objetivo	Cumplir especificaciones técnicas	Satisfacer necesidades del usuario
Método	Revisión técnica y pruebas internas	Pruebas funcionales y reales
Ejemplo	Revisar reglas mal formadas	Detectar errores de diagnóstico

Actividades Comunes de Verificación.

- ▶ **Revisión de conocimiento:**
 - ▶ Detección de reglas redundantes o contradictorias.
 - ▶ Verificación de completitud y consistencia en la base de conocimientos.
- ▶ **Validación formal de reglas:**
 - ▶ Comprobación de la sintaxis (estructura lógica correcta).
 - ▶ Uso de lógica proposicional para garantizar que las reglas no violan restricciones formales.
- ▶ **Evaluación del motor de inferencia:**
 - ▶ Correcta aplicación de encadenamiento hacia adelante o hacia atrás.
 - ▶ Manejo adecuado de incertidumbre y contradicciones.
- ▶ **Pruebas automatizadas:**
 - ▶ Casos de prueba con entradas conocidas y resultados esperados.
 - ▶ Simulación de escenarios críticos o extremos.

Actividades Comunes de Validación.

Comparación con expertos humanos:

- ▶ Se mide el grado de concordancia, por ejemplo: mediante
 - ▶ la **precisión** (Accuracy)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

donde: TP = True Positives, FP = False Positives, TN = True Negatives y FN = False Negatives.

- ▶ la **tasa de concordancia** (Cohen's Kappa)

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

donde: P_o = Proporción de acuerdo observado y
 P_e = Proporción de acuerdo esperado por azar.

$$P_e = \sum_i \left(P_i^{(1)} \times P_i^{(2)} \right)$$

Donde: $P_i^{(j)}$ = proporción de la categoría i según el evaluador j , $j = 1, 2$.

Evaluación en condiciones reales:

- ▶ Probar el sistema con datos reales del dominio y casos representativos.

Retroalimentación de usuarios:

- ▶ encuestas, entrevistas y estudios de usabilidad.

Validación empírica:

- ▶ Uso de conjuntos de prueba independientes (cross-validation¹).
- ▶ Medición con métricas como: precision, recall, F1-score.

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

¹Técnica de evaluación que divide los datos en varios subconjuntos para entrenar y validar un modelo repetidamente, con el fin de estimar su capacidad de generalización en datos no vistos.

Métodos de Verificación y Validación.

- ▶ **Revisión por expertos:**

Evaluación manual de reglas y lógica para detectar errores y inconsistencias.

- ▶ **Pruebas con casos:**

Uso de escenarios diseñados para verificar el comportamiento esperado del sistema.

- ▶ **Cross-validation:**

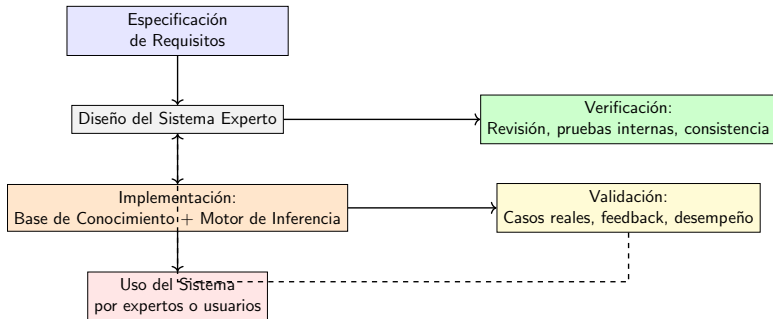
Técnicas estadísticas para estimar la precisión y generalización, como la validación cruzada k-fold:

$$\text{Error}_{cv} = \frac{1}{k} \sum_{i=1}^k \text{Error}_i$$

- ▶ **Métodos formales:**

Aplicación de análisis lógico y matemático para garantizar propiedades como consistencia y completitud.

Proceso de Verificación y Validación.



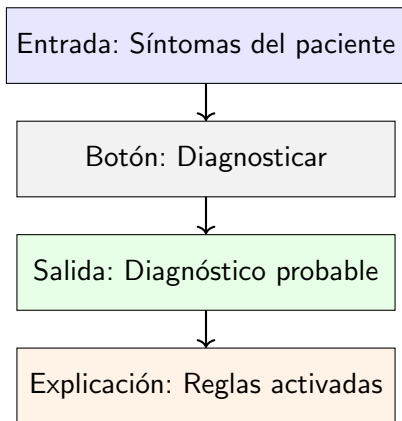
Importancia del Diseño de Interfaces.

- ▶ La **interfaz de usuario** es el canal de comunicación entre el sistema experto y el usuario final.
- ▶ Un **diseño deficiente puede anular la utilidad** de un sistema técnicamente correcto.
- ▶ El diseño debe estar alineado con el **modelo mental del usuario** y ser específico del dominio.

Elementos clave de la interfaz:

- ▶ **Entrada de datos:** formularios estructurados, asistentes interactivos, validación dinámica.
- ▶ **Salida de conocimiento:** decisiones razonadas, gráficos explicativos, niveles de confianza (p. ej. barras de probabilidad).
- ▶ **Módulo explicativo:** botones o menús que permitan consultar “¿Por qué esta decisión?” o “¿Qué pasaría si...?”.

Ejemplo de Interfaz de Usuario.

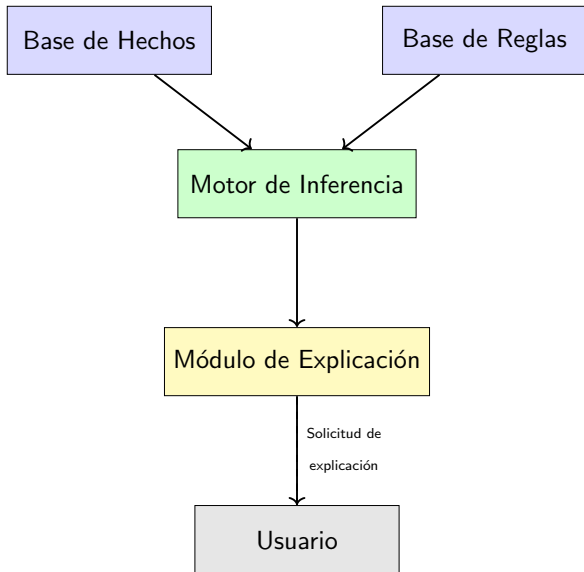


¿Qué es un Módulo de Explicación?

- ▶ Es un subcomponente crítico del sistema experto responsable de:
 - ▶ **Explicar el razonamiento:** cómo se llegó a una conclusión (trazabilidad).
 - ▶ **Justificar decisiones:** por qué se aplicó una regla y no otra.
- ▶ Incrementa la **transparencia** y la **confianza** del usuario.
- ▶ Es fundamental para:
 - ▶ **Auditorías:** revisión del razonamiento.
 - ▶ **Depuración:** localización de errores lógicos o reglas inconsistentes.
 - ▶ **Entrenamiento:** aprendizaje del conocimiento implícito.

Importancia del Diseño de Interfaces

Esquema funcional del módulo:



Tipos de Explicaciones.

- ▶ **Explicación hacia atrás (Why):** Justifica por qué se llegó a una conclusión específica.
- ▶ **Explicación hacia adelante (How):** Detalla cómo se construyó el razonamiento desde los hechos hasta la conclusión.
- ▶ **Explicación hipotética (What if):** Explora consecuencias de modificar ciertos hechos o condiciones.

Ejemplo lógico:

Regla: $A \wedge B \rightarrow C$

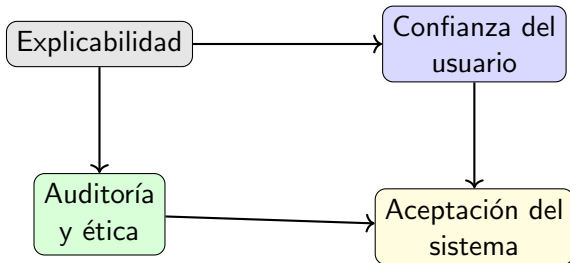
$$A = 1, \quad B = 1 \Rightarrow C = 1$$

- ▶ **Why:** “¿Por qué se concluyó C ?”
Porque se verificó A y B .
- ▶ **How:** “¿Cómo se llegó a C ?”
Se evaluó $A \wedge B$ como verdadero y se aplicó la regla.
- ▶ **What if:** “¿Qué pasaría si $B = 0$?”
Entonces $A \wedge B = 0$, y C no podría deducirse.

Importancia de la Explicabilidad.

- ▶ La **explicabilidad** es fundamental para generar **confianza**, **transparencia** y **aceptación** del sistema experto.
- ▶ Permite al usuario:
 - ▶ **Justificar las decisiones** del sistema.
 - ▶ **Detectar errores**, inconsistencias o sesgos.
 - ▶ **Aprender del razonamiento** experto (uso educativo).
- ▶ Es clave para:
 - ▶ **Auditoría** y trazabilidad en dominios críticos (medicina, derecho).
 - ▶ **Ética** y responsabilidad en decisiones automatizadas.
 - ▶ **Cumplimiento de regulaciones** (p. ej., *derecho a explicación* en GDPR).

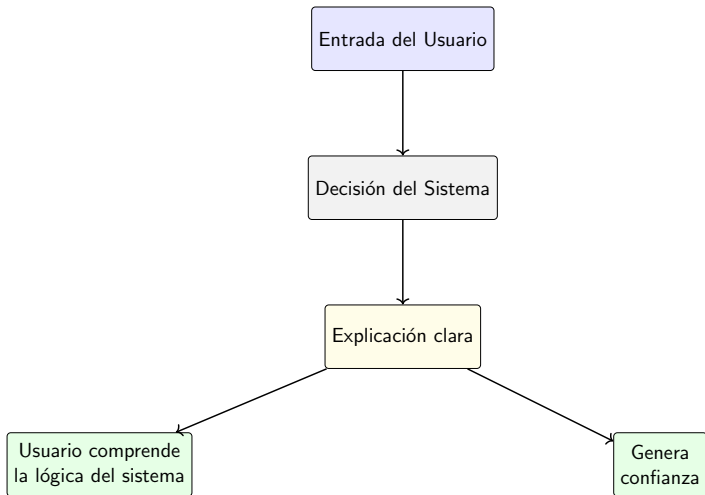
Relación entre Explicabilidad y Confianza.



Modelo Cognitivo de Confianza.

- ▶ La confianza del usuario en un sistema experto depende de su capacidad para:
 - ▶ **Comprender las decisiones tomadas** por el sistema.
 - ▶ **Evaluar la lógica detrás** de dichas decisiones.
 - ▶ **Confirmar que el razonamiento es coherente** con el conocimiento del dominio.
- ▶ Una buena explicación actúa como intermediario entre la decisión y la confianza.

Importancia del Diseño de Interfaces



- ▶ Una **interfaz bien diseñada** es esencial para facilitar la interacción entre el usuario y el sistema experto.
Debe ser:
 - ▶ **Intuitiva** y adaptada al conocimiento del usuario.
 - ▶ **Clara en la presentación** de entradas, salidas y razonamientos.
- ▶ El **módulo de explicación** permite:
 - ▶ **Justificar decisiones** tomadas por el sistema.
 - ▶ **Aumentar la trazabilidad** y facilitar auditorías.
 - ▶ **Detectar errores o inconsistencias** en el conocimiento.
- ▶ La **confianza del usuario** está determinada por:
 - ▶ La **calidad de las explicaciones** generadas.
 - ▶ La capacidad del usuario de **entender el razonamiento del sistema**.
 - ▶ La consistencia entre el sistema y la experiencia del usuario.

Sistemas Híbridos.

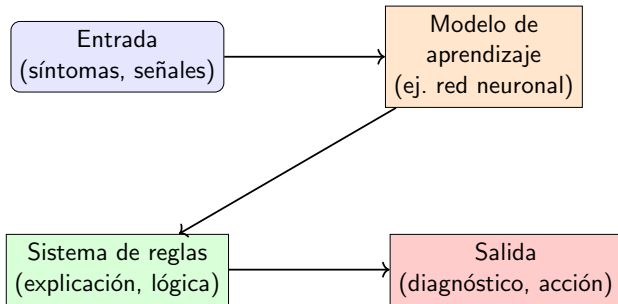
- ▶ Combinan **conocimiento simbólico** (reglas, lógica) con **técnicas estadísticas** (aprendizaje automático).
- ▶ Objetivo: combinar la **capacidad de aprendizaje** de los métodos estadísticos con la **capacidad explicativa** y controlable del conocimiento simbólico.
- ▶ Aplicaciones: diagnóstico médico, control industrial, sistemas financieros inteligentes, vehículos autónomos, etc.

- ▶ **Datos de Entrada:** Representan la información cruda proveniente del entorno, sensores o bases de datos. Pueden ser estructurados o no estructurados.
- ▶ **Modelo de ML (aprendizaje automático):** Se encarga de procesar los datos utilizando técnicas como redes neuronales, árboles de decisión o SVM. Produce predicciones o clasificaciones con base en patrones aprendidos de datos históricos.
- ▶ **Motor de Inferencia (sistema experto):** Aplica reglas lógicas del tipo “SI-ENTONCES” sobre los datos, usando conocimiento explícito de expertos humanos. Ofrece una interpretación simbólica del problema.
- ▶ **Fusión de Resultados: Combina las salidas** del modelo de ML y del sistema experto.
- ▶ **Decisión Final:** Resultado integrado que representa la mejor acción o diagnóstico basado en el análisis combinado de datos y conocimiento.

Ejemplo modelo secuencial:

- Una red neuronal detecta patrones → genera hipótesis diagnósticas → un sistema basado en reglas valida, explica o corrige.

$$\text{Decisión final} = f_{\text{reglas}}(f_{\text{ML}}(\mathbf{x}))$$



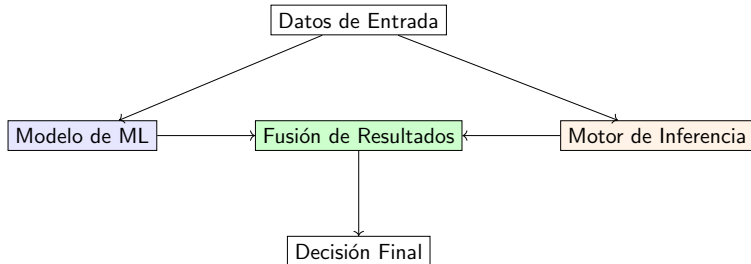
Ejemplo modelo paralelo:

- ▶ **Fusión de Resultados:** Combina las salidas del modelo de ML y del sistema experto. Métodos comunes incluyen:
 - ▶ Combinación lineal ponderada:

$$D = \alpha \cdot D_{ML} + (1 - \alpha) \cdot D_{SE}$$

donde $\alpha \in [0, 1]$ controla el peso relativo.

- ▶ Votación o reglas lógicas de consenso.



Ejemplo Aplicado: Diagnóstico Médico Asistido

- ▶ **Entrada (Input):** Conjunto de síntomas clínicos reportados por el paciente, por ejemplo: fiebre, tos, fatiga.
- ▶ **Modelo de Aprendizaje Automático (ML):**
 - ▶ Una red neuronal convolucional (CNN) entrenada con historias clínicas sugiere una distribución de probabilidad sobre enfermedades posibles.
 - ▶ Salida típica:

$$P_{ML} = [\text{Gripe: } 0,6, \text{ COVID-19: } 0,3, \text{ Neumonía: } 0,1]$$

- ▶ **Sistema Experto (Reglas Lógicas):**
 - ▶ Utiliza reglas del tipo “Si fiebre Y tos seca ENTONCES COVID-19”.
 - ▶ Salida simbólica convertida a probabilidades:

$$P_{reglas} = [\text{Gripe: } 0,3, \text{ COVID-19: } 0,6, \text{ Neumonía: } 0,1]$$

► Fusión de Resultados (Combinación Híbrida):

$$P_{\text{final}} = \alpha \cdot P_{\text{ML}} + (1 - \alpha) \cdot P_{\text{reglas}}, \quad \alpha \in [0, 1]$$

- Controla el balance entre aprendizaje automático y reglas humanas.
- Permite tunear la explicabilidad frente a la adaptabilidad.

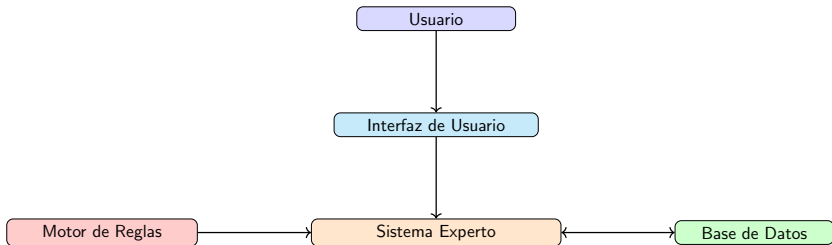
Ventaja:

- **Precisión** de modelos entrenables +
- **Explicabilidad** de sistemas simbólicos.

Integración con Bases de Datos.

- ▶ Los sistemas expertos requieren **información precisa y actualizada** para emitir conclusiones fiables.
- ▶ Las **bases de datos** permiten acceder a grandes volúmenes de datos estructurados, organizados por tablas y relaciones.
- ▶ El **acoplamiento con bases de datos** permite al sistema experto:
 - ▶ Consultar hechos actualizados en tiempo real.
 - ▶ Reducir la entrada manual de datos.
 - ▶ Mejorar la coherencia y consistencia del conocimiento.
- ▶ Este vínculo mejora la escalabilidad del sistema y facilita su integración con otras aplicaciones empresariales.

Esquema de Integración con BD.



Ejemplo de razonamiento con datos estructurados.

- ▶ **Regla del sistema experto:**

SI morosidad > 6 meses ENTONCES riesgo = alto

- ▶ **Consulta SQL ejecutada:**

*SELECT * FROM clientes WHERE morosidad > 6;*

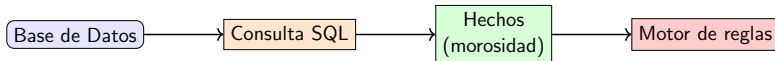
- ▶ **Integración con el motor de inferencia:**

- ▶ Los resultados se traducen a hechos como:

`morosidad(cliente123) = 8`

- ▶ El sistema experto activa reglas con esos hechos:

`riesgo(cliente123) := alto`



Características de Sistemas en Tiempo Real.

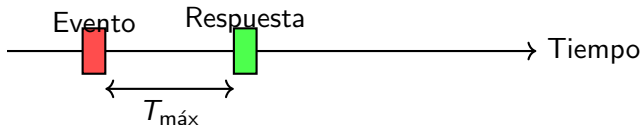
- ▶ **Reacción temporal:** deben responder a eventos en un tiempo máximo permitido $T_{\text{máx}}$.

$$t_{\text{respuesta}} \leq T_{\text{máx}}$$

- ▶ **Eficiencia y concurrencia:** múltiples procesos se ejecutan simultáneamente para cumplir con los límites temporales.
- ▶ **Robustez:** capacidad de operar correctamente ante fallos y cargas variables.
- ▶ **Monitoreo continuo:** vigilancia constante del sistema y su entorno para detectar eventos críticos.
- ▶ **Decisiones rápidas:** algoritmos optimizados para inferencia y control en tiempo real.

Ejemplos típicos:

- ▶ Control de tráfico aéreo — evita colisiones y mantiene rutas seguras.
- ▶ Diagnóstico en maquinaria industrial — detecta y previene fallos críticos en segundos.



Ejemplo: Sistema de Monitoreo de Temperatura.

Regla en el sistema experto:

Si $T > 80^{\circ}\text{C} \Rightarrow \text{activar(alarma)}$

Ejemplo de ejecución:

1. Sensor mide: $T = 85^{\circ}\text{C}$
2. El hecho `temperatura(85)` se inserta en la base de hechos.
3. Regla activada por el motor de inferencia.
4. Se genera señal de alarma en menos de $T_{\text{máx}}$ milisegundos.



Propósito: responder a condiciones críticas en tiempo real, protegiendo sistemas sensibles al calor.

Conclusiones.

► Integración del conocimiento.

Los sistemas expertos modernos requieren combinar diferentes fuentes y formas de representación del conocimiento (reglas, lógica, datos).

► Potencia de los sistemas híbridos.

La fusión de técnicas simbólicas (reglas, lógica) con métodos estadísticos (ML, inferencia probabilística) permite:

- Alta precisión en la toma de decisiones.
- Explicabilidad y trazabilidad de los resultados.

► Conectividad e implementación.

Conectar sistemas expertos con bases de datos y sensores en tiempo real:

- Permite aplicaciones en contextos críticos (industria, salud).
- Requiere eficiencia, concurrencia y robustez en la ejecución.

Tendencias y Futuro de los Sistemas Expertos

- ▶ Las tendencias en sistemas expertos se orientan hacia **arquitecturas híbridas** que integran conocimiento explícito y aprendizaje automático, **combinando razonamiento simbólico y capacidad de adaptación basada en datos**.
- ▶ Destacan:
 - ▶ los sistemas basados en **ontologías** para una representación semántica más estructurada,
 - ▶ el impulso de la **Inteligencia Artificial explicable (XAI)** para garantizar transparencia, y
 - ▶ el desarrollo de **sistemas multi-agente y distribuidos** que permiten mayor escalabilidad y cooperación en entornos complejos.

Sistemas basados en conocimiento y aprendizaje automático.

- ▶ Los sistemas híbridos combinan:
 - ▶ **Representación explícita de conocimiento** (reglas, ontologías).
 - ▶ **Aprendizaje a partir de datos** mediante modelos como redes neuronales.
- ▶ **Ventajas:**
 - ▶ Explicabilidad mediante trazabilidad lógica.
 - ▶ Adaptación dinámica a nuevos patrones de datos.
- ▶ **Aplicación práctica:** diagnóstico médico con:
 - ▶ Sistema experto que codifica el conocimiento clínico.
 - ▶ Clasificador aprendido con datos históricos.

Modelo de fusión probabilística:

$$P(\text{diagnóstico} \mid \text{datos}) = \alpha \cdot P_{\text{ML}} + (1 - \alpha) \cdot P_{\text{reglas}}, \quad \alpha \in [0, 1]$$

Ontologías.

- ▶ Una **ontología** es una especificación formal y explícita de una conceptualización compartida de un dominio.
- ▶ Define:
 - ▶ **Conceptos** (clases, entidades)
 - ▶ **Relaciones** entre conceptos
 - ▶ **Axiomas** o restricciones lógicas
- ▶ Facilita la **interoperabilidad**, la **reutilización** y la **consistencia** del conocimiento.

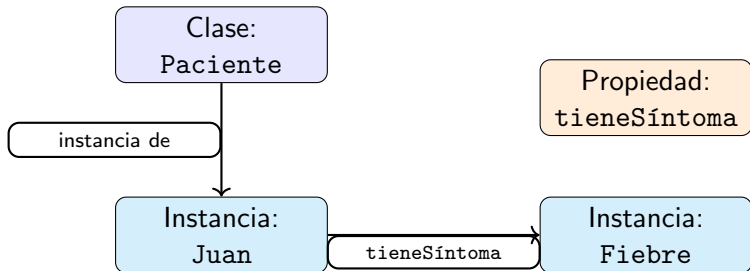
Ejemplo: Ontología médica (fragmento)

Clase: Paciente, Propiedad: tieneSíntoma, Instancia: Juan

tieneSíntoma(Juan, Fiebre)

Aplicaciones: Web Semántica, diagnóstico médico, bioinformática, derecho.

Ejemplo: Ontología médica (fragmento).



Inteligencia Artificial Explicable (XAI: Explainable Artificial Intelligence).

- ▶ **Transparencia:** las decisiones de los sistemas inteligentes deben ser comprensibles para humanos.
- ▶ **Confianza:** los usuarios deben poder verificar y justificar el comportamiento del sistema.
- ▶ **Explicabilidad:** XAI combina modelos predictivos con técnicas de interpretación y visualización.

¿Por qué es necesario XAI?

- ▶ Regulaciones como GDPR² exigen “derecho a explicación”.
- ▶ Impacto crítico en dominios sensibles: medicina, finanzas, justicia.

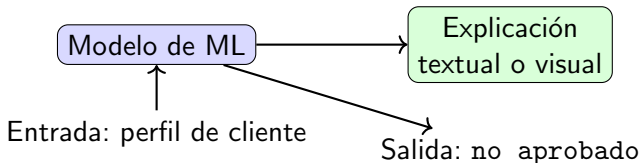
²Reglamento General de Protección de Datos (por sus siglas en inglés, General Data Protection Regulation).

Ejemplo :

Un cliente fue rechazado para un crédito.

XAI permite responder:

‘‘El ingreso mensual fue bajo y la antigüedad laboral < 1 año.’’



Métodos de XAI.

- ▶ **Modelos intrínsecamente interpretables:**
 - ▶ Árboles de decisión, reglas “if-then”, regresión lineal.
 - ▶ La explicación forma parte del modelo en sí.
- ▶ **Métodos post-hoc:**
 - ▶ Analizan un modelo ya entrenado (*caja negra*) para explicar sus predicciones sin modificar su estructura.
 - ▶ Técnicas destacadas:
 - ▶ **LIME** (Local Interpretable Model-agnostic Explanations): aproxima localmente el modelo con uno interpretable (por ejemplo, regresión lineal).
 - ▶ **SHAP** (SHapley Additive exPlanations): asigna a cada característica una contribución basada en la teoría de juegos de Shapley.
 - ▶ El modelo se trata como una función $f(x)$, y se busca descomponer su salida como: $f(x) = \phi_0 + \sum_{i=1}^n \phi_i$ donde ϕ_0 es el valor base (promedio) y ϕ_i es la contribución de la característica i .

Ejemplo: valor SHAP:

- ▶ Basado en teoría de juegos: asigna a cada característica su contribución justa a la predicción.
- ▶ La predicción se expresa como:

$$f(x) = \phi_0 + \sum_{i=1}^n \phi_i$$

- ▶ $\phi_0 = 0,50$: predicción base (promedio del modelo sin información del individuo).
 - ▶ $\phi_1 = +0,10$: historial crediticio bueno.
 - ▶ $\phi_2 = -0,25$: ingreso bajo.
 - ▶ $\phi_3 = -0,15$: deuda alta.
- ▶ **Resultado:** $f(x) = 0,50 + 0,10 - 0,25 - 0,15 = 0,20 \rightarrow$ No aprobar crédito

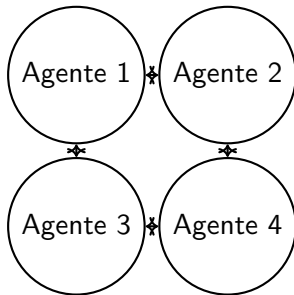


Sistemas Multi-Agente y Distribuidos.

Definición de Agente y Sistema Multi-Agente.

- ▶ **Agente:** entidad autónoma con capacidad de:
 - ▶ **Percibir** su entorno a través de sensores.
 - ▶ **Actuar** sobre el entorno mediante actuadores.
 - ▶ **Tomar decisiones** de forma autónoma.
- ▶ **Sistema Multi-Agente (MAS):**
 - ▶ Conjunto de agentes que interactúan.
 - ▶ Pueden **cooperar**, **competir** o **negociar**.
 - ▶ Se comunican mediante protocolos definidos.

Sistema Multi-Agente.



Ejemplos: robótica cooperativa, trading algorítmico, gestión distribuida del tráfico.

Sistema Distribuido.

- ▶ Es un conjunto de componentes computacionales que se comunican y coordinan a través de una red para alcanzar un objetivo común.
- ▶ Cada componente actúa de manera concurrente y puede estar ubicado en diferentes nodos físicos.

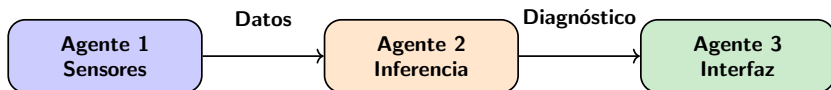
Ventajas del enfoque multi-agente distribuido:

- ▶ **Escalabilidad:** fácil de extender con nuevos agentes o módulos.
- ▶ **Tolerancia a fallos:** el sistema puede seguir funcionando aunque un agente falle.
- ▶ **Modularidad:** cada agente se puede desarrollar y mantener por separado.
- ▶ **Descentralización:** evita puntos únicos de fallo y mejora la eficiencia.

Ejemplo de diagnóstico distribuido.

Sistema Multi-Agente aplicado al diagnóstico clínico:

- ▶ **Agente 1 (Adquisición):** recoge datos de sensores (temperatura, frecuencia cardíaca, etc.).
- ▶ **Agente 2 (Inferencia):** aplica reglas o modelos probabilísticos para determinar posibles diagnósticos.
- ▶ **Agente 3 (Interfaz):** comunica los resultados al usuario o profesional médico.



Conclusiones.

- ▶ El **futuro de los sistemas expertos** radica en su capacidad de integración con:
 - ▶ **Aprendizaje automático (ML)** para adaptabilidad.
 - ▶ **Ontologías** para una representación estructurada del conocimiento.
 - ▶ **XAI (Inteligencia Artificial Explicable)** para transparencia.
 - ▶ **Tecnologías distribuidas y multi-agente** para escalabilidad y robustez.
- ▶ **Tendencias clave:**
 - ▶ Mayor demanda de sistemas que justifiquen sus decisiones.
 - ▶ Requerimientos de modularidad y tolerancia a fallos.
- ▶ **Ámbitos de aplicación** actuales y futuros:
 - ▶ Medicina personalizada y diagnósticos asistidos.
 - ▶ Automatización industrial y mantenimiento predictivo.
 - ▶ Toma de decisiones en políticas públicas y servicios gubernamentales.

Herramientas Libres para Desarrollo de SE.

- ▶ Las herramientas libres para el desarrollo de Sistemas Expertos (SE) **han facilitado la creación, implementación y experimentación** con sistemas basados en conocimiento sin necesidad de licencias propietarias.
- ▶ Entre ellas se destacan **entornos como CLIPS, Jess y Drools**, que permiten definir reglas, representar conocimiento y construir mecanismos de inferencia basados en lógica.
- ▶ Estas herramientas **favorecen la investigación, la enseñanza y el desarrollo** de aplicaciones inteligentes, promoviendo la interoperabilidad y la integración con otras tecnologías de inteligencia artificial.

CLIPS (C Language Integrated Production System).

- ▶ Desarrollado por la NASA.
- ▶ Entorno popular para construir sistemas expertos.
- ▶ Lenguaje propio, estilo Lisp, fácil de aprender.
- ▶ Soporta:
 - ▶ Reglas de producción (if-then)
 - ▶ Programación orientada a objetos (COOL)
- ▶ Muy rápido y bien documentado.
- ▶ **Licencia:** Pública



CLIPS

A Tool for Building Expert Systems

Developed at NASA's Johnson Space Center from 1985 to 1996, the C Language Integrated Production System (CLIPS) is a rule-based programming language useful for creating expert systems and other programs where a heuristic solution is easier to implement and maintain than an algorithmic solution. Written in C for portability, CLIPS can be installed and used on a wide variety of platforms. Since 1996, CLIPS has been available as public domain software.

[DOWNLOAD](#)

[DOCUMENTATION](#)

[SUPPORT](#)

[Link al WEB.](#)

Jess (Java Expert System Shell).

- ▶ Similar a CLIPS, pero escrito en Java.
- ▶ Lenguaje propio basado en CLIPS.
- ▶ Integración completa con aplicaciones Java.
- ▶ Ampliamente usado en entornos académicos.
- ▶ **Licencia:** No completamente libre (licencias académicas disponibles).

Davidson
Campen (C-4)

DAVIDSON 4.020 (Lanada)
Univision Release
First Printed November 1987

Jess, The Java Expert System Shell

<http://www.cba.hawaii.edu/jess/>
David F. Davidson (Ed)
Davidson Computing Systems
Stanley Systems Laboratory
Livermore, CA
Version 5.061 (11 September, 1999)

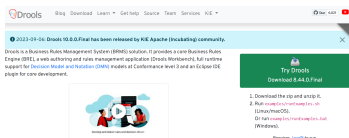
ABSTRACT

This expert shell is an expert system shell and scripting language written entirely in Java. Microsoft's Java language has supported the development of rule-based expert systems which can be tightly coupled to code written in the powerful, portable, Java language. The spirit of the Java language is dynamic, and a comprehensive list of supported functions is presented. Goals in calling Java functions from Java, to extending Java by writing Java code, and to embedding Java in Java applications are also included.

[Link al WEB.](http://www.cba.hawaii.edu/jess/)

Drools.

- ▶ Motor de reglas de negocio basado en Java.
- ▶ Lenguaje DRL (Drools Rule Language).
- ▶ Soporta:
 - ▶ Sistemas expertos complejos
 - ▶ Integración con BPM
- ▶ Fuerte soporte comunitario y corporativo.
- ▶ **Licencia:** Apache 2.0



[Link al WEB.](#)

Prolog (GNU Prolog / SWI-Prolog).

- ▶ Lenguaje lógico clásico para sistemas expertos.
- ▶ Ideal para razonamiento simbólico y búsqueda.
- ▶ Flexibilidad total pero con curva de aprendizaje.
- ▶ **Licencias:** GPL / BSD.

The GNU Prolog web site



Current stable version is gprolog-1.5.0

[Link a GNU Prolog.](#)



[Link a SWI-Prolog.](#)

OpenL Tablets.

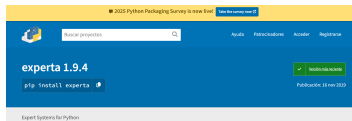
- ▶ Define reglas utilizando hojas de cálculo estilo Excel.
- ▶ Enfocada a usuarios no técnicos.
- ▶ Basada en Java, fácil integración empresarial.
- ▶ **Licencia:** Apache 2.0.



[Link al WEB.](#)

Experta.

- ▶ Biblioteca moderna en **Python** para construir sistemas expertos basados en reglas.
- ▶ Inspirada en CLIPS, con una sintaxis declarativa y elegante en Python.
- ▶ Compatible con versiones recientes de Python.
- ▶ Proyecto activo con mantenimiento y mejoras continuas.
- ▶ **Licencia:** MIT.



[Link al WEB.](#)

[Link al GitHub.](#)

Taller: Sistema Experto con Factores de Certeza para Verificación de Piezas Mecánicas.

Objetivo.

- ▶ Diseñar e implementar un sistema experto capaz de verificar diferentes tipos de piezas mecanizadas (eje, brida, tornillo), evaluando si cumplen con sus especificaciones técnicas.
- ▶ El sistema debe manejar la **incertidumbre** mediante **factores de certeza**, acumulando evidencias parciales de fallas usando la técnica del sistema MYCIN.

Materiales Suministrados.

Descripción de tipos de piezas con sus parámetros característicos:

- ▶ Eje: diámetro, longitud, rugosidad, material, grietas visibles.
- ▶ Brida: diámetro, longitud, rugosidad, material, grietas visibles.
- ▶ Tornillo: diámetro, longitud, rugosidad, material, grietas visibles.

Tolerancias Técnicas por Tipo de Pieza.

Tipo	Diámetro (mm)	Longitud (mm)	Rugosidad máx (Ra)	Material
Eje	49.8 – 50.2	99.7 – 100.3	2.0	Acero
Brida	80.0 – 80.5	19.5 – 20.5	2.0	Aluminio
Tornillo	9.8 – 10.2	19.0 – 21.0	1.5	Acero

Factores de Certeza por Evidencia.

Condición detectada	FC asociado
Diámetro fuera de tolerancia	0.4
Longitud fuera de tolerancia	0.3
Rugosidad excesiva	0.3
Material incorrecto	0.6
Grietas visibles	0.7

Fórmula de Combinación (MYCIN).

$$FC_{\text{total}} = FC_1 + (1 - FC_1) \cdot FC_2$$

Actividades del Taller.

1. Implementar una función para evaluar cada pieza en función de sus medidas y las tolerancias.
2. Aplicar factores de certeza para cada evidencia de fallo detectada.
3. Implementar la función para combinar evidencias siguiendo la fórmula de MYCIN.
4. Para cada pieza, mostrar:
 - ▶ El **factor de certeza acumulado** de rechazo.
 - ▶ Las **razones** específicas de posible fallo.
 - ▶ Un mensaje final que indique si la pieza debe ser aceptada o rechazada.

Gracias por su atención.

